Høyskolen Kristiania

Institutt for Teknologi, innovasjon og økonomi – frontend og mobilutvikling

Eksamen

Emne: Android Programmering

Emnekode: PGR208

Semester: høst, 2025.

Dato for innlevering: 02. Desember, 2025.

Gruppenummer: 36

Kandidatnummer: 125, 111, 130

Undervisere: Rolando Gonzalez

Innholdsfortegnelse

INTRODUKSJON **2**

APPEN INNEHOLDER FØLGENDE SKJERMER: 2
METODER OG TEKNOLOGIER 3
SKJERM 1 – ANIME-LISTE 3
SKJERM 2: SØK ETTER ANIME VED HJELP AV ID-SKJERMBILDER 5
SKJERM 3: EGNE ANIME-IDEER (ROM) 7
SKJERM 4 – MATCHUP (SPILLFUNKSJONALITET) 10
PLANLEGGINGS- OG SKRIVEPROSESS 12
RESSURSER 12
KONKLUSJON 13

Introduksjon

AnimeApp er en Android-programmeringsapplikasjon utviklet i PGR208. Appen består av fire skjermer og utnytter moderne Android-teknologier som Jetpack Compose, MVVMarkitektur, Retrofit, Room Database, Navigation Compose og Coil for bildefremvisning. Vårt mål har vært å skape en app som er ren, stabil og brukervennlig, med fokus på tilstandshåndtering og ren kode. Vi er tre personer i utviklingsteamet, og på grunn av eksamens kravene måtte hver skjerm tilby mer enn det som først ble spesifisert. Dette oppnådde vi ved å forbedre brukeropplevelsen gjennom ekstra logikk, validering, utvidet visning av informasjon og interaktivitet.

Appen inneholder følgende skjermer:

1. Anime-bibliotek hentet fra API/ Vise alle animer

2. Søkemeny der brukeren kan søke etter anime basert på ID

3. Side basert på Room hvor brukere kan lagre sine egne anime-ideer

4. Matchup-side hvor deltakerne spiller et lite spill basert på popularitet

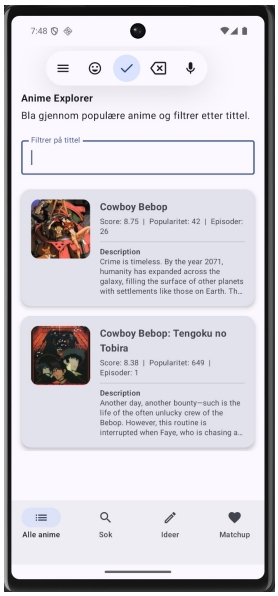
Gjennom prosjektet har vi arbeidet med arkitektur, integrering med API-er, oppbygging av databaser, navigasjonsfunksjoner og Compose. Appen er pålitelig, implementert med en ren struktur, og demonstrerer ferdighetene vi har tilegnet oss i løpet av kurset på en produktiv måte.

Metoder og teknologier

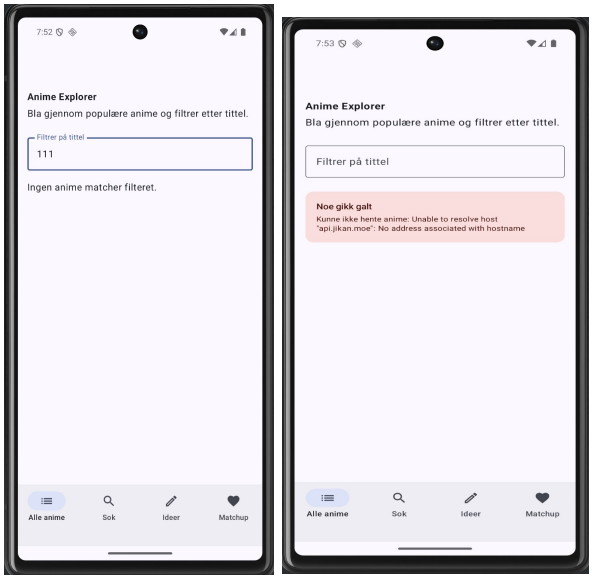
- Jetpack Compose: All brukergrensesnitt er utviklet i Compose, som gir et moderne og deklarativt UI som oppdateres dynamisk ved endringer i ViewModel-tilstanden.
- MVVM: Hver skjerm har en egen ViewModel som håndterer logikk, API-anrop og kommunikasjon med databasen, noe som bidrar til en ren og enkel UI-kode.
- Retrofit + Jikan API: Vi benytter Retrofit for å hente både lister og instanser av anime fra Jikan API, med hjelp av GsonConverterFactory og OkHttp LoggingInterceptor.
- Room Database: Room brukes til lokal lagring av brukerens anime-ideer, og er fullt implementert med Entity, DAO, Database og Repository.
- Navigation Compose: Navigasjonen i appen er organisert med NavHost og fire destinasjoner, der vi bruker en bunnlinje for rask navigering.
- StateFlow og UiState-modeller: Lasting, data og feil håndteres i UiState på alle skjermer, noe som gir appen stabilitet og forutsigbarhet.
- Coil: Coil brukes for rask og Compose-vennlig bildefylling fra API-et.

Skjerm 1 – Anime-liste

(SKJERMBILDE 1: Skjerm 1 – Laster tilstand



(SKJERMBILDE 2: Skjerm 1 – Fullastet liste med anime-kort)

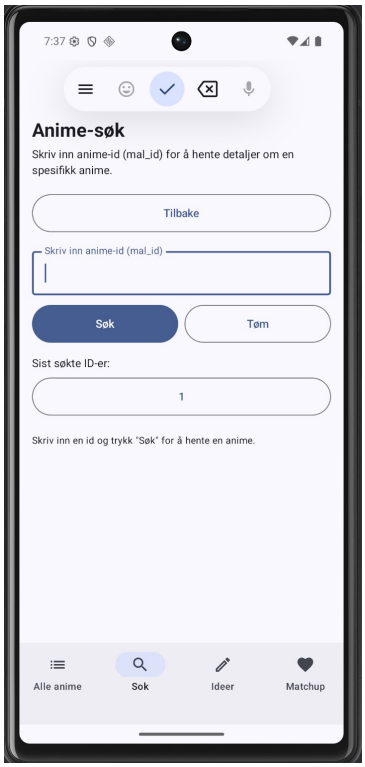


(SKJERMBILDE 3: Skjerm 1 – Feilmelding fra API)

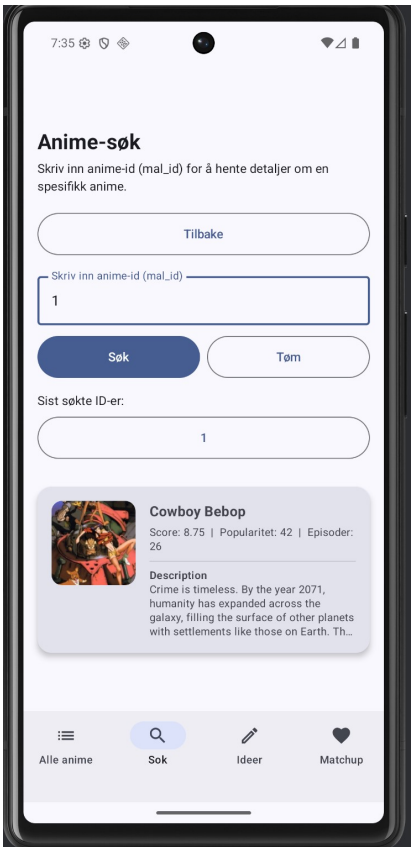
Skjerm 1 viser anime fra Jikan API. AnimeListViewModel håndterer data fra AnimeRepository, mens Retrofit brukes til API-anrop. ViewModelen opprettholder en UiState-modell som inkluderer isLoading, errorMessage og animeList. En "Laster..."indikator vises først. Ved feil fra API-et eller manglende respons, viser vi en rød feilmelding via vår ErrorMessage-komponent. Vellykkede anrop vises i en LazyColumn, der objektene for hver anime presenteres gjennom en AnimeListItem eller AnimeCard-modul. Hver kort inkluderer bilde, tittel, ID, poengsum, popularitet, episoder og en innholdssammendrag. For å forbedre brukeropplevelsen har vi inkludert sanntidsfiltrering av anime. Søkeområdet øverst lar brukeren skrive inn tekst, og listen filtreres automatisk basert på dette. Vi har implementert

lastetilstand, feilhåndtering og en mer omfattende kortoppsett enn minimumskravet. I utviklingen erfarte vi viktigheten av en klar UiState-modell, spesielt siden Jikan API kan returnere ulike feil. Dette resulterte i mer tilpasset feilvisning og sterkere logging ved bruk av OkHttp. Vi lærte også at filtrering bør foregå i ViewModel for å sikre en rask og ryddig UI. Denne skjermen fungerer som appens front-end og gir et helhetlig og attraktivt inntrykk.

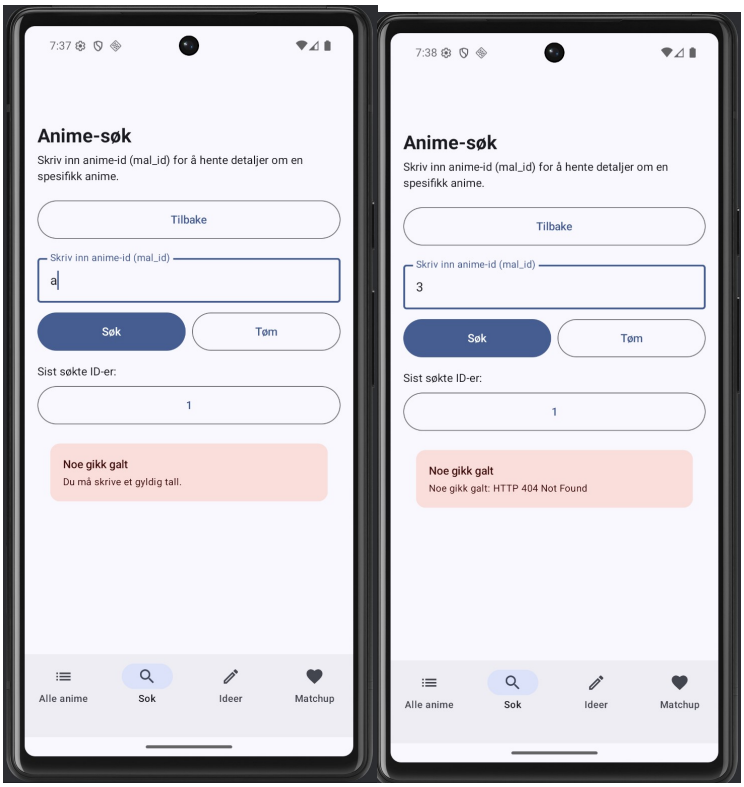
Skjerm 2: Søk etter anime ved hjelp av ID-skjermbilder



(SKJERMBILDE 4: Søkeområde for anime-ID)



(SKJERMBILDE 5: Resultatet vises)



(SKJERMBILDE 6: Feilmeldinger og tom tilstand)

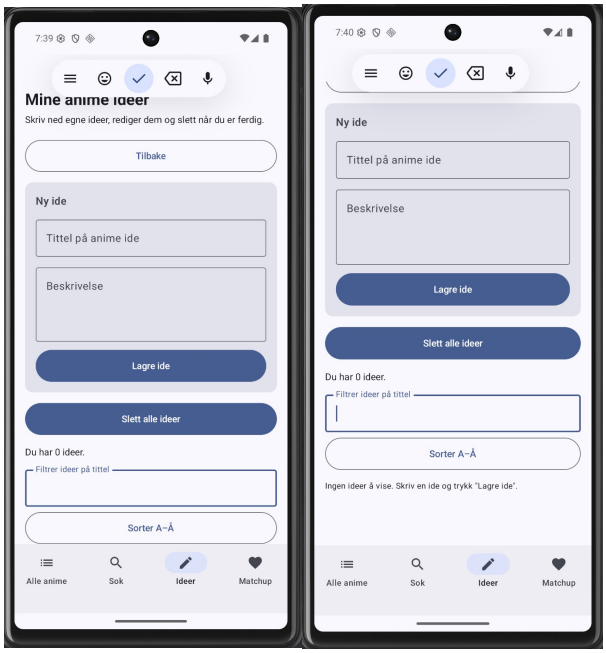
På skjerm 2 kan brukeren søke etter anime ved å skrive inn ID-en. Vi lagrer tilstanden for lasting, resultat, feilmeldinger og en historikk over de siste vellykkede søkene ved hjelp av AnimeSearchViewModel og AnimeSearchUiState. Først validerer vi inndataene ved å bruke toIntOrNull() når brukeren trykker på "Søk". Hvis resultatet er et ugyldig nummer, får brukeren en feilmelding: "Du må skrive inn et gyldig nummer," uten at API-et blir kalt.

Når vi har en gyldig ID, setter vi isLoading = true, og deretter henter vi dataene fra Jikan API gjennom AnimeRepository. Hvis API-et finner animeen, oppdateres UiState med denne informasjonen. Skjermen viser deretter et AnimeCard med bilde, tittel, popularitet, poengsum, episoder og synopsis. Vi lagrer også ID-en i en liten historikk, som vises som hurtigknapper i søkefeltet, slik at brukeren kan klikke på en tidligere ID for å søke på nytt.

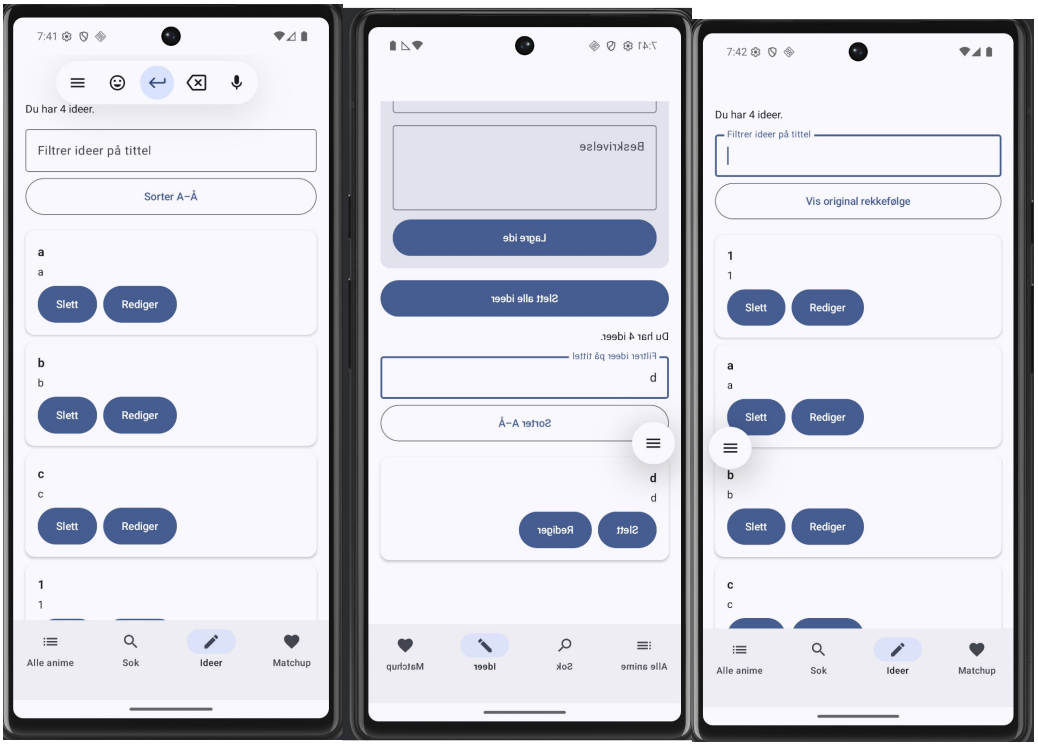
I tillegg finnes det en Fjernknapp som tilbakestiller resultatet og feilmeldingene til en ren, tom tilstand. Hvis det oppstår feil i API-et eller nettverket, vises også en feilmelding med en kort forklaring på årsaken. Vi har tre nivåer av feilhåndtering, ugyldig inndata, teknisk feil og tomt resultat. Under utviklingen har vi møtt lignende problemer med nullverdier, da ikke alle feltene alltid har blitt returnert fra Jikan API. Dette løste vi med sikre kall (?.) i komponentene.

Skjerm 2 går utover det grunnleggende ved å validere inndataene, laste inn data, bruke hurtigknapper for sist søk, ha en fjernknapp, og sørge for at informasjonen er lett tilgjengelig.

Skjerm 3: Egne anime-ideer (Rom)



(SKJERMBILDE 7: Tom liste)



(SKJERMBILDE 8: Liste med lagrede ideer, filtrert og sortert)

(SKJERMBILDE 9: Redigeringsvisning med valideringsfeil)

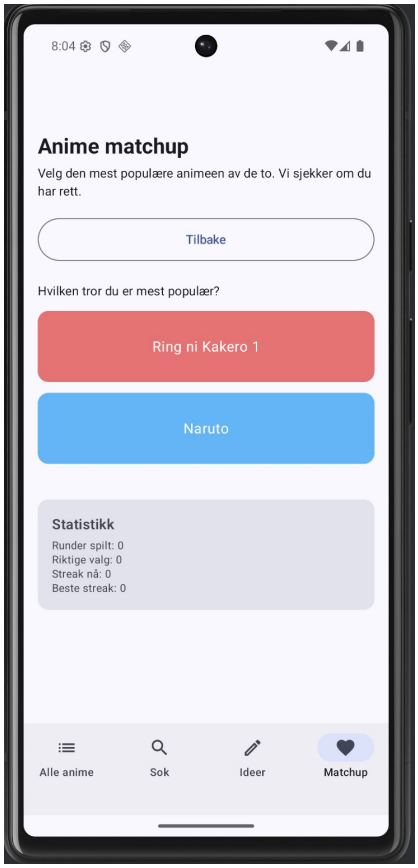
Skjerm 3 bruker Rom for å lagre brukerens egne anime-ideer. Vi tilbyr AnimeIdea (Entity), AnimeIdeaDao (DAO), AppDatabase (Database) og AnimeRepository i den komplette databaseimplementeringen. ViewModel AnimeIdeasViewModel gir sanntidsoppdateringer til UI ved hjelp av StateFlow og AnimeIdeasUiState, som inkluderer en liste over ideer, tekstfelt for tittel og beskrivelse, lastetilstand og feilmeldinger.

Brukeren kan opprette nye ideer ved å skrive inn en tittel og beskrivelse, og deretter trykke på "Lagre ide." Hvis feltene er tomme, setter ViewModel en feilmelding i UiState, som vises direkte under inntastingskortet. Ideene vises i en LazyColumn, der hver ide har knappene "Slett" og "Rediger". Når brukeren velger å redigere, fylles feltene automatisk med den eksisterende tittelen og beskrivelsen, og knappen endres til "Oppdater ide." Dette gir en klar redigeringsmodus og en bedre brukeropplevelse enn bare å lagre og slette.

Videre har vi lagt til en knapp for å "Slette alle ideer", en teller som forteller "Du har X ideer", et filterfelt, og muligheten til å filtrere listen alfabetisk (A–Z) eller gå tilbake til den opprinnelige listen. Dette forenkler idestyringen og gir brukeren mer kontroll.

Gjennom utviklingen har vi også oppdaget verdien av å kjøre databaseoperasjoner i bakgrunnstråder med korutiner, samt hvordan StateFlow opprettholder UI-synkronisering med databasen. Skjerm 3 er en av de mest teknisk krevende skjermene i appen, spesielt med tanke på det ekstra kravet om funksjonalitet for en gruppe på tre.

Skjerm 4 – Matchup (spillfunksjonalitet)



(SKJERMBILDE 10: Startvisning med teksten "Velg den mest populære anime" og To tilfeldige anime vises)

(SKJERMBILDE 12: Tilbakemelding på om brukeren valgte riktig)

Matchup-skjermen er vår tilpassede skjerm, hvor brukeren får presentert to tilfeldige anime fra API-en. Basert på poengsum og popularitetsrangering skal brukeren velge hvilken anime som er mest populær. ViewModel AnimeMatchupViewModel henter en liste over anime, trekker ut to tilfeldige, og oppdaterer AnimeMatchupUiState med valgt par, resultater, meldinger og statistikk. Brukeren kan deretter klikke på et av anime-kortene for å sammenligne poengsummen mot populariteten. ViewModel oppdaterer:

- lastChosenTitle
- comparisonMessage (beskriver hvorfor valget var riktig eller galt)
- lastWasMostPopular
- Statistikk: totalt antall runder, riktige valg, nåværende rekke og beste rekke

UI-en viser deretter et kort med resultatet, og et nytt par anime velges automatisk. Dette gir en spillaktig flyt og gir skjermen betydelig verdi. Som tilleggsfunksjoner har vi inkludert:

- Spillmekanikk
- Automatisk rundetelling
- Rekkesystem

- Beste rekke
- Dynamisk tilbakemelding
- Egen seksjon for statistikk
- Feilhåndtering og lasting

Selvom det bare er nødvendig å lage en "tilpasset skjerm med funksjonalitet og interaksjon relevant for temaet", valgte vi og utviklet et komplett minispill. Koden er solid med ryddig UiState-objektdesign og god kontroll over tilstanden. Skjerm 4 demonstrerer vår evne til å kombinere Compose, ViewModels og API-logikk på en effektiv måte.

Planleggings- og skriveprosess

Vi startet prosjektet ved å lese gjennom eksamensoppgaven og diskutere hvordan vi skulle organisere arbeidet. Det var viktig for oss, som et team på tre, at hver skjerm hadde funksjoner som støttet hverandre. Vi laget en enkel, men effektiv arbeidsplan der vi fordelte skjermene mellom oss, men samarbeidet også om API, Room og UiState, og jobbet sammen på kodebasen. Vi sjekket ofte inn med hverandre og benyttet en oppgaveliste tidlig i prosessen for å holde oss organisert. Under utviklingen brukte vi emulatoren til testing og logging for å løse API-anrop, nullverdier og Room-problemer. Vi hjalp hverandre gjennom utfordringer, spesielt med ViewModels og UI-overganger. En nøkkelbeslutning var å bruke UiState-klasser for hver skjerm, noe som reduserte kompleksiteten og risikoen for redundans i den komponerbare funksjonen. Vi prioriterte gjenbrukbare komponenter som AnimeCard, ErrorMessage, LoadingIndicator og knappkomponenter for å opprettholde en konsistent UI. Vi lærte mye mer om Room, spesielt om hvordan redigering skal fungere og hvordan man oppdaterer databasen uten å måtte navigere på nytt i appen. Til slutt sørget vi for at appen var stabil og brukervennlig før vi begynte på rapporten, noe som gjorde at vi kunne skrive om den faktiske funksjonaliteten til koden fremfor teorien. Totalt sett har dette vært en verdifull læringsopplevelse som har styrket vårt samarbeid med fokus på implementering, versjonskontroll, API, Compose og lokal datalagring.

Ressurser

- Offisiell Android-dokumentasjon
- Jikan API-dokumentasjon
- Pensum, forelesningsnotater og tidligere uker i PGR208

- Vi har brukt AI som et støtteverktøy for å hjelpe med å forklare akademiske konsepter, for å klargjøre eventuelle tvil underveis, og for å forbedre formuleringer i rapportspråk. AI ble kun brukt som veiledning, og alt teknisk innhold, kode og refleksjoner er utviklet og kvalitetssikret av oss selv (OpenAI, 2025).

Konklusjon

AnimeApp oppfyller alle eksamenskrav, inkludert tilleggskravene for en gruppe på tre. Vi har utviklet en app som utnytter moderne Android-teknologier, implementerer riktig arkitektur, og tilbyr en god brukeropplevelse med relevante funksjoner. Vi er stolte av resultatet og mener at applikasjonen demonstrerer vår mestring av Jetpack Compose, MVVM, Room, Retrofit og tilstandshåndtering på en profesjonell måte.

Referanseliste

Google. (2024). *Jetpack Compose documenta0on*. Android Developers. hBps://developer.android.com/jetpack/compose

Google. (2024). *Guide to app architecture*. Android Developers. hBps://developer.android.com/topic/architecture

Google. (2024). *ViewModel overview*. Android Developers. hBps://developer.android.com/topic/libraries/architecture/viewmodel

Google. (2024). *State and Jetpack Compose*. Android Developers. hBps://developer.android.com/jetpack/compose/state

Google. (2024). *Naviga0on for Compose*. Android Developers. hBps://developer.android.com/jetpack/compose/navigaLon

Google. (2024). *Save data in a local database using Room*. Android Developers. hBps://developer.android.com/training/data-storage/room

Square. (2024). *Retrofit: A type-safe HTTP client for Android and Java*. hBps://square.github.io/retrofit/

Coil. (2024). *Coil Compose documenta0on*. hBps://coil-kt.github.io/coil/compose/

Jikan Project. (2024). *Jikan REST API*. hBps://docs.api.jikan.moe/

JetBrains. (2024). *Kotlin documenta0on*. hBps://kotlinlang.org/docs/home.html

Google. (2024). *Kotlin coroutines on Android*. Android Developers. https://developer.android.com/kotlin/coroutines

Google. (2024). *Material Design 3 guidelines*. hBps://m3.material.io/

OpenAI. (2024). ChatGPT (Version GPT-4.1) [Large language model]. hBps://chat.openai.com